

AI Resume Checker Workflow

This document explains how the project works from startup to user actions. The project is a resume analysis and candidate ranking app with a React frontend, a Python FastAPI backend, MongoDB storage, and NLP ranking logic.

1. Project Structure

ai resume chacker/ package.json	Root scripts for running frontend and backend together
frontend/ src/App.jsx	React + Vite application Route setup
src/context/ResumeContext.jsx	Shared user and resume analysis state
src/pages/ src/components/Navbar.jsx	Main app pages Navigation
backend/ main.py	API and persistence layer Active FastAPI backend used by root scripts
database.py	MongoDB connection helpers
models.py	Pydantic schemas and MongoDB serialization
nlp_engine.py	PDF extraction, preprocessing, TF-IDF ranking
server.js	Older/alternate Express backend
notebook/ report/	Resume screening experiment notebook Existing generated reports and report builders

2. How To Run

Install dependencies first if they are missing:

```
npm install
cd frontend
npm install
cd ../backend
pip install -r requirements.txt
```

Start MongoDB locally:

```
mongod
```

Run the full app from the project root:

```
npm run dev
```

This starts:

```
Frontend: http://localhost:5173
Backend: http://localhost:5000
```

The root `package.json` runs `frontend` with Vite and `backend/main.py` with Uvicorn.

3. Application Workflow

User Account Flow

1. User opens the React frontend.

2. User signs up from `/signup`.
3. `Signup.jsx` sends `POST http://localhost:5000/api/signup`.
4. `backend/main.py` checks MongoDB for an existing email.
5. The backend hashes the password with `bcrypt`.
6. A new user document is saved in the `users` collection.
7. User logs in from `/login`.
8. `Login.jsx` sends `POST /api/login`.
9. The backend verifies the password and returns user profile data.
10. `ResumeContext.jsx` stores the user in React state and `localStorage`.

Single Resume Analysis Flow

1. User visits `/upload`.
2. `UploadResume.jsx` accepts a PDF/DOCX resume and optional job description.
3. User clicks `Analyze Resume`.
4. The app navigates to `/loading` and passes the file, job description, user id, email, and password in route state.
5. `LoadingScreen.jsx` reads the file in the browser as base64.
6. It sends the resume and prompt directly to the Gemini API.
7. Gemini returns JSON with:
 - ATS score
 - job match score
 - recruiter score
 - missing skills
 - suggestions
 - chart data
 - keyword density data
 - improved resume bullet suggestions
8. `LoadingScreen.jsx` stores that response in `ResumeContext`.
9. It also sends the resume file and analysis result to `POST /api/upload`.
10. `backend/main.py` stores the resume file bytes and parsed analysis in MongoDB.
11. The frontend navigates to `/result`.
12. `ResultPage.jsx` displays the score dashboard, resume preview, missing keywords, skill chart, and AI suggestions.

Candidate Ranking Flow

1. User visits `/ranking`.
2. `RankingPage.jsx` accepts multiple resumes plus a job description.
3. User clicks Rank Candidates.
4. The frontend sends a multipart request to POST `http://localhost:5000/api/rank`.
5. `backend/main.py` reads each uploaded resume.
6. `nlp_engine.extract_text_from_pdf()` extracts text from PDFs using `pypdf`.
7. `nlp_engine.rank_candidates()` compares each resume against the job description.
8. The ranking algorithm:
 - preprocesses text with NLTK
 - removes URLs, emails, numbers, punctuation, and stopwords
 - stems words with PorterStemmer
 - builds TF-IDF vectors with unigrams and bigrams
 - computes cosine similarity
 - adds a skill overlap bonus
 - sorts candidates by final score
9. The backend returns ranked candidates.
10. `RankingPage.jsx` shows rank, score, matched skills, missing skills, detected skills, word count, education indicators, and experience indicators.

Dashboard And Saved Reports Flow

1. User visits `/dashboard`.
2. `Dashboard.jsx` calls GET `/api/resumes?userId=<id>`.
3. Backend returns saved resume summaries from MongoDB.
4. Dashboard calculates:
 - total uploaded resumes
 - average ATS score
 - average match score
 - predicted interviews based on score threshold
5. User can view a previous report.
6. `Dashboard.jsx` calls GET `/api/resumes/{id}`.
7. Backend returns full saved resume data.

8. Frontend places `parsedData` into `ResumeContext` and navigates to `/result`.

Profile And Settings Flow

1. `/profile` loads user data from `ResumeContext`.
2. It fetches user resumes with `GET /api/resumes?userId=<id>`.
3. User can:
 - edit name/title with `PUT /api/users/{id}`
 - view saved reports
 - download stored resume files
 - delete individual resumes with `DELETE /api/resumes/{id}`
4. `/settings` stores UI preferences in `localStorage`.
5. Dark mode is applied by adding/removing the `dark` class on `document.documentElement`.
6. Account deletion calls `DELETE /api/users/{id}`, which deletes the user and all linked resumes.

4. Backend API Workflow

Method	Endpoint	Purpose
POST	<code>/api/signup</code>	Create user account
POST	<code>/api/login</code>	Authenticate user
POST	<code>/api/upload</code>	Save resume file and analysis data
GET	<code>/api/resumes</code>	List saved resumes, optionally filtered by user id
GET	<code>/api/resumes/{id}</code>	Fetch one saved resume/report
DELETE	<code>/api/resumes/{id}</code>	Delete one saved resume
PUT	<code>/api/users/{id}</code>	Update user profile
DELETE	<code>/api/users/{id}</code>	Delete user and all resumes
POST	<code>/api/rank</code>	Rank multiple resumes against a job description
GET	<code>/api/rank/test</code>	Health check for ranking service

5. Data Flow Diagram

```
React UI
|
|  signup/login/upload/report/ranking requests
v
```

FastAPI backend on localhost:5000

|
| users and saved resume reports
v

MongoDB database: ai-resume-checker

Single resume analysis:

React Upload -> LoadingScreen -> Gemini API -> ResumeContext -> FastAPI upload
-> MongoDB -> Result page

Candidate ranking:

React RankingPage -> FastAPI /api/rank -> nlp_engine.py -> ranked response ->
RankingPage results

6. Important Notes

- backend/main.py is the active backend because root package.json runs python -m uvicorn main:app.
- backend/server.js is an alternate Express implementation with similar routes, but it is not used by the root dev script.
- The frontend has hard-coded backend URLs using http://localhost:5000.
- The single-resume ATS analysis currently calls Gemini directly from browser code in LoadingScreen.jsx.
- The Gemini API key is hard-coded in the frontend, which is not safe for production. It should be moved to the backend or an environment variable workflow.
- Candidate ranking currently extracts text only with pypdf, so uploaded DOCX files may not rank correctly unless DOCX extraction is added.